E

EDUSYNC

Project workflow and database guide

A visual, plain-language walkthrough of every major operation - with a deeper look at the MySQL data model.

THE BIG IDEA

Faculty create the academic structure, record sessions and marks, and review class insights. Students see their own progress. MySQL stores the underlying records; Express and Prisma read and update them; React presents the results.

WHAT YOU WILL UNDERSTAND

01 How the app and API fit together

02 What every database table represents

03 How attendance, assessment, and dashboards work

04 How to inspect data and safely prepare a demo

Built from the current EduSync source code | 24 October 2026

Contents

Follow the lifecycle from database setup through the student support loop.

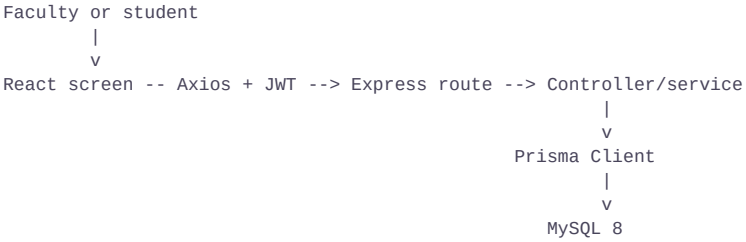
Contents	2
1. The short version	3
2. Start and sign in	3
3. Understand the academic database	3
4. Faculty setup: create the foundation first	5
5. Attendance: from a class meeting to a percentage	5
6. Assessments and marks: draft, save, publish	6
7. Dashboards, alerts, and intervention	6
8. Academic support assistant and data sharing	7
9. API map: screen operation to endpoint	7
10. How to inspect database records	8
11. Migrations, seed data, and safety	9
12. Implementation details and boundaries	9
13. Source files worth opening while learning	9
Glossary	10

This guide describes the current EduSync implementation: React + Vite in client/, Express in server/, Prisma in server/prisma/schema.prisma, and MySQL as the database. It supplements the project workflow and API documents with extra attention to what is stored, where it is stored, and how the parts connect.

1. The short version

EduSync manages courses over an academic term. Faculty set up sections, subjects, offerings, and students. For each offering, they record class sessions and attendance, then create assessments, enter marks, and publish results. The server reads those records to calculate progress, show alerts, and give students a dashboard. Faculty can add remarks for student interventions.

The database is the source of truth. React does not own a separate copy of academic records: screens request data from the Express API, and the API reads or writes records through Prisma.



Example: when faculty marks one student absent, the browser sends the session ID, student ID, and ABSENT to the API. The server validates the request, Prisma inserts or updates the matching attendance row, and a later dashboard request reads that saved row to recalculate attendance.

2. Start and sign in

From the repository root:

```
docker compose up -d mysql adminer
npm run db:migrate
npm run dev
```

Open http://localhost:5173. The API health check is http://localhost:3000/api/health; Adminer is available at http://localhost:8080 when the Compose services are running.

The server reads its database connection from DATABASE_URL in server/.env. Keep that file private; never put a real database password in this guide or commit it. Prisma reads the same connection setting from server/prisma/schema.prisma.

Demo sign-in (after the database has been prepared and seeded):

Role	Email	Password
Faculty	faculty@demo.com	Password@123
Student	student1@demo.com	Password@123

The login screen stores a signed token in browser local storage. Axios attaches it to protected API requests. Express verifies the token, and role middleware restricts faculty-only and student-only route groups.

3. Understand the academic database

The Prisma schema defines the data model; Prisma maps model names to the existing snake_case MySQL table and column names. For example, Prisma's passwordHash field maps to the password_hash database column. The model names in Prisma are singular (Student); the MySQL tables use plural names (students).

Relationship map

```
users (faculty) ------< offerings >------ subjects
                            |
                            +-------------- sections
                                                |
                                                +------< students >------ users (student)

offerings ------< class_sessions ------< attendance >------ students
offerings ------< assessments ---------< marks >----------- students
students ------< remarks >------ offerings
                    |
                    +---------------- users (faculty author)
```

< means "one record can be connected to many records." The arrows describe foreign-key relationships, not a separate copy of the linked record. For example, an attendance row stores a student_id; the student's name and email stay in users.

The 10 tables, in everyday language

MySQL table	What one row represents	Important links and rules
users	A sign-in identity for a faculty member or student	email is unique; role is FACULTY or STUDENT; password is stored as a bcrypt hash, never as the original password.
sections	A cohort in a semester and academic year	(name, semester, academic_year) must be unique. A section has students and course offerings.
students	A student's university number and section	user_id and usn are unique. Every student links to one login and one section.
subjects	A reusable course in the catalog	code is unique. Stores the course name and credits.
offerings	A subject taught to a section by a faculty member	Links subject_id, section_id, and faculty user_id. A subject-section pair can only appear once. This is the course workspace faculty open.
class_sessions	One scheduled class meeting for an offering	Links to an offering; stores date, period, topic, and OPEN/LOCKED state. The offering/date/period combination is unique.
attendance	One student's status for one class session	Links a session and a student; the session/student pair is unique. Status is PRESENT, LATE, or ABSENT.
assessments	One test, quiz, or assignment for an offering	Links to an offering; stores type, maximum marks, date, and DRAFT/PUBLISHED state.
marks	One student's score on one assessment	Links an assessment and student; that pair is unique. marks_obtained is a decimal and can be NULL.
remarks	A faculty note about a student in an offering	Links the student, course offering, and faculty author; keeps note text and creation time. Shown as a faculty note on the student's dashboard.

Keys, constraints, and why they matter

* Primary key (id): identifies one row. MySQL generates it. Related tables store this ID as a foreign key.
* Foreign key: prevents a row from referring to a parent row that does not exist. For example, a mark cannot refer to a nonexistent assessment.
* Unique constraint: prevents duplicate business records. The API also checks some duplicates to return friendlier error messages.
* Decimal marks: DECIMAL(5,2) preserves values such as 17.50 without binary floating-point rounding.

* Pair uniqueness: one attendance row per student/session and one marks row per student/assessment. The API uses Prisma upsert so saving a second time updates the existing record instead of creating a duplicate.
* Indexes: the schema indexes student IDs used for attendance and marks lookups.

Delete rules: parent rows can affect children

The schema explicitly cascades deletion for several relationships. Deleting a student user cascades to the student profile and that student's attendance, marks, and remarks. Deleting a subject or offering can cascade into offerings, sessions, attendance, assessments, marks, and remarks. Those are real data deletions, not just items hidden from the dashboard.

Sections are different: the student-to-section relationship is Restrict. MySQL can reject deleting a section while students still reference it. Before deleting academic data, inspect the dependencies and keep a backup. The UI asks for confirmation, but confirmation does not make a cascading delete reversible.

4. Faculty setup: create the foundation first

Use Term setup in this order:

* Sections -- Create the cohort, semester, and academic year. A section is the student group, such as Section A, Semester 5, 2026-2027. Create writes one row to sections.
* Subjects -- Create each course once in the catalog. Create writes one row to subjects, with a unique course code.
* Students -- Add one student or import several. Each student operation creates a users login plus a linked students profile. The CSV screen sends JSON rows (USN, Name, Email, and the selected sectionId) to the Express endpoint; it does not upload a file. New accounts currently use the shared starter password Password@123.
* Offerings -- Assign a subject to a section. This creates an offerings row and associates the current faculty account with it. The subject hub, sessions, and assessments all belong to this offering.

Each screen reads data with GET, creates it with POST, edits it with PUT, and deletes it with DELETE. Search and section filters are display filters; the records remain in MySQL. The current setup screens use edit and delete dialogs/actions for these records.

Why the order matters: offerings reference both a subject and a section; students reference a section. Create those parent records before assigning students or courses.

5. Attendance: from a class meeting to a percentage

Create a session

From a subject hub, create a session with a date, period, and topic. The server inserts one class_sessions row with status OPEN. It also initializes an attendance row as PRESENT for every student currently enrolled in the offering's section.

Edit and save the roster

The attendance sheet is assembled from the offering's current section roster, joined to the session's saved attendance rows. A student who has no row is shown as PRESENT by the sheet endpoint. For each changed status, the faculty screen sends student IDs and statuses. Prisma upsert inserts or updates attendance.

The three statuses mean:

* PRESENT -- counted as attended.
* LATE -- also counted as attended.
* ABSENT -- not counted as attended.

Saving a draft changes attendance records but does not lock the session. The attendance update route rejects changes once the session is LOCKED.

Lock or reopen the session

* Save & Lock saves the currently selected statuses and updates class_sessions.status to LOCKED.
* A locked session is included in attendance statistics and student dashboards.
* Reopen changes the status back to OPEN, so the attendance sheet can be edited again.

The lock and reopen endpoints update the status column; the attendance percentage is calculated when a dashboard or statistics endpoint reads the data. It is not a percentage column stored on each student.

Attendance calculation

For one student in one offering:

```
attendance % = (PRESENT rows + LATE rows) / LOCKED sessions * 100
```

Open sessions do not count. If there are no locked sessions, the stats service reports 100% and zero classes needed. The current default target used by the classes-needed calculation is 75%.

Example: 7 attended classes out of 10 locked sessions is 70%. The minimum number of future classes to attend to reach 75% is calculated by solving (7 + X) / (10 + X) >= 0.75; here, X = 2.

The offering's faculty statistics average per-student percentages. A student's overall dashboard combines attendance counts across their offerings, which is weighted by class sessions.

6. Assessments and marks: draft, save, publish

* Create an assessment -- Adds an assessments row in DRAFT, attached to an offering. Type is IA, ASSIGNMENT, or QUIZ.
* Open the marks sheet -- The server reads students in the offering's section and any matching marks rows. This produces one row per student.
* Save marks -- The API upserts one marks row per submitted student/assessment pair. Scores must be between zero and the assessment's maximum.
* Publish -- Changes assessment status to PUBLISHED. Only published assessments are returned as academic results to the student dashboard and used in published-mark statistics.

Marks have two important meanings:

* 0 is a real score of zero.
* NULL means no score was recorded, commonly because the student was absent. The performance calculation skips null marks and does not add that assessment's maximum to the denominator.

Implementation detail to know

The marks-sheet endpoint supports NULL, and the marks API can store it. However, the current React marks modal displays a null mark as 0 and submits it as a number when saving. In that screen, leaving the default value unchanged records zero rather than absence. Faculty who need absence semantics should not interpret a displayed zero as NULL.

The service calculates subject performance as:

```
performance % = sum(non-NULL marks) / sum(maximum marks for those graded assessments) * 100
```

Example: marks of 8/20 and 9/10 produce 17/30 = 56.67%. A null mark for another assessment does not change either total.

7. Dashboards, alerts, and intervention

Dashboards are live summaries calculated from the records described above:

* Faculty overview reads the faculty's offerings and computes attendance and at-risk summaries.
* Subject hub reads offering details and per-offering statistics, session lists, assessments, and intervention insights.

* Student dashboard finds the student's section and subjects, reads locked-session attendance and published marks, then includes applicable alerts and faculty remarks.
* Charts render values returned by the API; Chart.js does not calculate or persist the academic record.

The alert service compares live statistics with configured server thresholds:

Alert	Current rule
LOW_ATTENDANCE	Below ATTENDANCE_WARN (default 75%); critical below ATTENDANCE_CRITICAL (default 65%).
LOW_MARKS	Published assessment performance below MARKS_LOW (default 40%), provided performance is greater than zero.
DECLINING_TREND	At least three graded assessments, with each of the latest two lower than the one before it.

Faculty can add a remark from the subject's Insights view. The API inserts a remarks row with student, offering, faculty author, and note. The student dashboard reads those remarks. Alerts and stats are computed on request; the schema has no separate alerts table.

8. Academic support assistant and data sharing

The student can send a question to POST /api/me/assistant. The server first reads the current student's offerings, locked-session attendance, and published marks from MySQL and prepares a compact academic snapshot.

* Without a configured AI key, or when the student does not opt in, the application answers with local progress-based guidance.
* AI-provider replies require both OPENAI_API_KEY on the server and explicit opt-in (shareProgress: true) for that request.
* The provider request contains the student's question plus an academic snapshot (subject code, percentages, and classes needed); it does not include the student's email, name, or USN.
* The API does not write assistant chats to the database.

This assistant is a study-support feature, not an authoritative academic policy or attendance exception service.

9. API map: screen operation to endpoint

All routes below are mounted under /api. Protected calls include Authorization: Bearer <token>. Sections, subjects, offerings, students, sessions, and assessments require a faculty role. /me/dashboard and /me/assistant require a student role.

Operation	Endpoint	Main database change or read
Sign in	POST /auth/login	Reads users, compares the password hash, returns a signed token.
Current identity	GET /auth/me	Reads the authenticated users row and optional student profile.
Change password	POST /auth/change-password	Updates users.password_hash after checking the old password.
Sections	GET/POST/PUT/DELETE /sections	Reads or changes sections.
Subjects	GET/POST/PUT/DELETE /subjects	Reads or changes subjects.
Students	GET/POST/PUT/DELETE /students	Reads/changes students and linked users; delete cascades.
Student import	POST /students/import	Creates student login/profile rows from JSON; response includes inserted count and row errors.

Operation	Endpoint	Main database change or read
Offerings	GET/POST/PUT/DELETE /offerings	Reads or changes offerings; creates an offering from subject + section + faculty.
Offering details	GET /offerings/:id	Reads the course, section, roster, sessions, and assessments.
Offering statistics	GET /offerings/:id/stats	Reads attendance and marks to calculate class averages and at-risk count.
At-risk students	GET /offerings/:id/at-risk	Reads attendance, published marks, and remarks for the section roster.
Create/list sessions	POST/GET /offerings/:id/sessions	Inserts class_sessions and initial attendance, or reads sessions.
Attendance sheet	GET /sessions/:sid/sheet	Reads roster and attendance rows for one session.
Save attendance	PUT /sessions/:sid/attendance	Upserts rows in attendance; only while session is open.
Lock/reopen session	PUT /sessions/:sid/lock, POST /sessions/:sid/reopen	Changes class_sessions.status.
Create/list assessments	POST/GET /offerings/:id/assessments	Inserts a draft in assessments, or reads offering assessments.
Edit/delete/publish assessment	PUT/DELETE /assessments/:aid, POST /assessments/:aid/publish	Updates/deletes the assessment or changes status.
Read/save marks	GET/PUT /assessments/:aid/marks	Reads roster + marks, or upserts marks.
Student dashboard	GET /me/dashboard	Reads a student's current records and calculates summary, alerts, and remarks.
Student coach	POST /me/assistant	Reads progress for a reply; does not persist chat history.
Read/add remarks	GET/POST /students/:id/remarks	Reads or inserts remarks.

Successful API responses generally use { "success": true, "data": ... }. Errors return success: false and a message. HTTP 401 usually means missing/expired token; 403 means wrong role; 404 means no matching record; 400 means invalid input or operation.

10. How to inspect database records

Adminer provides a graphical interface: open http://localhost:8080, connect to the MySQL service, choose the attendance_tracker database, and browse tables. Use values from your local Docker Compose and .env files to connect; credentials are not repeated in this PDF.

Read-only SQL examples (adjust the MySQL host/database as needed):

```
-- See the academic foundation.
SELECT id, name, semester, academic_year
FROM sections;

SELECT id, code, name, credits
FROM subjects;

-- Follow a course offering to its subject and cohort.
SELECT o.id, s.code, s.name, sec.name AS section_name, sec.semester
FROM offerings AS o
JOIN subjects AS s ON s.id = o.subject_id
JOIN sections AS sec ON sec.id = o.section_id;

-- See the attendance statuses for one session.
SELECT cs.session_date, u.name, st.usn, a.status
FROM attendance AS a
```

```
JOIN class_sessions AS cs ON cs.id = a.session_id
JOIN students AS st ON st.id = a.student_id
JOIN users AS u ON u.id = st.user_id
WHERE cs.id = 1;

-- See grades and whether an assessment is published.
SELECT u.name, st.usn, asm.title, asm.status,
       m.marks_obtained, asm.max_marks
FROM marks AS m
JOIN students AS st ON st.id = m.student_id
JOIN users AS u ON u.id = st.user_id
JOIN assessments AS asm ON asm.id = m.assessment_id;
```

1 in the session query is an example ID, not a guaranteed ID. Start with the session table and use an ID that exists in your local database. Do not run DELETE, TRUNCATE, or DROP queries to "reset" a shared database.

11. Migrations, seed data, and safety

* schema.prisma describes what tables and relationships the app expects. A migration applies schema changes to MySQL. Review schema changes and back up important data before applying them.
* npm run db:migrate runs Prisma's development migration command. This repository currently has a Prisma schema file but no checked-in migration directory; database initialization may require Prisma to create the first migration interactively.
* npm run db:seed runs server/prisma/seed.js. Do not run it against a database that contains data you need. The current script deletes existing remarks, marks, attendance, assessments, sessions, offerings, students, subjects, sections, and users before inserting demo records.
* Current seed limitation: this version of seed.js calls seedCN(...) and seedSE(...), but those helper functions are not defined in the file. As written, the seed can delete existing data, insert part of the demo data, then fail. Only run the seed on a disposable development database after fixing or verifying the seed script.
* Keep .env out of Git. Use .env.example as a variable-name reference and place real connection strings/passwords in the ignored local environment file.

12. Implementation details and boundaries

* Prisma is the persistence layer; the app does not use MongoDB, PostgreSQL, SQLite, or a separate document store.
* Statistics and alerts are calculated when requested, not stored in dedicated summary or alert tables. A chart refresh can therefore show newly saved attendance/marks without a separate aggregation job.
* Role middleware protects faculty and student route groups. The route definitions currently shown apply role checks; the existence of ownership.js alone does not prove every endpoint checks that a faculty member owns the requested offering. Do not treat object-level faculty ownership as verified without checking the individual controller/middleware path.
* When a student is added to a section, they appear in the section roster used for future attendance sheets and assessment sheets. Existing sessions/assessments are not backfilled by a separate enrollment-history table.
* Changing or deleting parent academic records can affect child records due to MySQL foreign keys and cascade settings. Back up before destructive operations.
* This is a development/hackathon guide. For production, review seed behavior, authorization/ownership checks, password onboarding, backups, migrations, and deployment settings before using real student records.

13. Source files worth opening while learning

File	What to look for
server/prisma/schema.prisma	The source of truth for MySQL entities, relationships, enums, and constraints.
server/src/routes/index.js	API prefix and resource router registration.

File	What to look for
server/src/routes/scaffoldRoutes.js	Endpoints and role guards.
server/src/controllers/sessionController.js	Attendance sheet reads, attendance upserts, lock/reopen state.
server/src/controllers/assessmentController.js	Marks reads/upserts, score validation, publish state.
server/src/controllers/offeringController.js	Offering setup, sessions, assessment creation, and faculty stats.
server/src/controllers/meController.js	Student dashboard reads and computed response.
server/src/services/statsService.js	Attendance and mark calculation formulas.
server/src/services/alertService.js	Alert thresholds and declining-trend rules.
server/src/services/assistantService.js	Progress snapshot and consent-based AI/local coach.
server/prisma/seed.js	Demo records; inspect before ever executing because it resets data.

Glossary

* Offering: one subject assigned to one section and a faculty member. It is the container for that course's sessions, assessments, and remarks.
* Roster: students currently linked to a section.
* Upsert: update a row if its unique key already exists; otherwise insert it.
* Foreign key: a stored ID that links a child record to its parent.
* Nullable / NULL: a database field may intentionally have no value; it is different from numeric zero.
* Migration: a versioned database schema change.
* Seed: a script that populates demo data. In this repository's current implementation it also deletes data first.